

visibility. You don't create a `XamlRoot` object directly. Instead, one is created when you attach a XAML element to an `AppWindow`. You can then use the `UIElement.XamlRoot` property to retrieve the `XamlRoot`.

For more info about `UIContext` and `XamlRoot`, see [Make code portable across windowing hosts](#).

Show a new window

Let's take a look at the steps to show content in a new `AppWindow`.

To show a new window

1. Call the static `AppWindow.TryCreateAsync` method to create a new `AppWindow`.

C#

```
AppWindow appWindow = await AppWindow.TryCreateAsync();
```

2. Create the window content.

Typically, you create a XAML `Frame`, then navigate the `Frame` to a XAML `Page` where you've defined your app content. For more info about frames and pages, see [Peer-to-peer navigation between two pages](#).

C#

```
Frame appWindowContentFrame = new Frame();  
appWindowContentFrame.Navigate(typeof(AppWindowMainPage));
```

However, you can show any XAML content in the `AppWindow`, not just a `Frame` and `Page`. For example, you can show just a single control, like `ColorPicker`, or you can show a `SwapChainPanel` that hosts DirectX content.

3. Call the `ElementCompositionPreview.SetAppWindowContent` method to attach the XAML content to the `AppWindow`.

C#

```
ElementCompositionPreview.SetAppWindowContent(appWindow,  
appWindowContentFrame);
```

The call to this method creates a `XamlRoot` object and sets it as the `XamlRoot` property for the specified `UIElement`.

You may only call this method once per `AppWindow` instance. After the content has been set, further calls to `SetAppWindowContent` for this `AppWindow` instance will fail. Also, if you attempt to disconnect the `AppWindow` content by passing in a null `UIElement` object, the call will fail.

4. Call the `AppWindow.TryShowAsync` method to show the new window.

C#

```
await appWindow.TryShowAsync();
```

Release resources when a window is closed

You should always handle the `AppWindow.Closed` event to release XAML resources (the `AppWindow` content) and references to the `AppWindow`.

C#

```
appWindow.Closed += delegate
{
    appWindowContentFrame.Content = null;
    appWindow = null;
};
```

Tip

You should keep the amount of code in your `Closed` event handler to the minimum amount possible to avoid unexpected issues.

Track instances of AppWindow

Depending on how you use multiple windows in your app, you may or may not need to keep track of the instances of `AppWindow` you create. The `HelloAppWindow` example shows some different ways you might typically use an `AppWindow`. Here, we'll look at why these windows should be tracked, and how to do it.

Simple tracking

The color picker window hosts a single XAML control, and the code for interacting with the color picker all resides in the `MainPage.xaml.cs` file. The color picker window only

allows a single instance and is essentially an extension of `MainWindow`. To ensure that only one instance is created, the color picker window is tracked with a page level variable. Before creating a new color picker window, you check whether an instance exists, and if it does, skip the steps to create a new window and just call `TryShowAsync` on the existing window.

C#

```
AppWindow colorPickerAppWindow;

// ...

private async void DetachColorPickerButton_Click(object sender,
RoutedEventArgs e)
{
    // Create the color picker window.
    if (colorPickerAppWindow == null)
    {
        // ...
        // Create a new window
        colorPickerAppWindow = await AppWindow.TryCreateAsync();
        // ...
    }
    // Show the window.
    await colorPickerAppWindow.TryShowAsync();
}
```

Track an AppWindow instance in its hosted content

The `AppWindowPage` window hosts a complete XAML page, and the code for interacting with the page resides in `AppWindowPage.xaml.cs`. It allows multiple instances, each of which functions independently.

The functionality of the page lets you manipulate the window, setting it to `FullScreen` or `CompactOverlay`, and also listens for `AppWindow.Changed` events to show information about the window. In order to call these APIs, `AppWindowPage` needs a reference to the `AppWindow` instance that is hosting it.

If that's all that you need, you can create a property in `AppWindowPage` and assign the `AppWindow` instance to it when you create it.

AppWindowPage.xaml.cs

In `AppWindowPage`, create a property to hold the `AppWindow` reference.

C#

```
public sealed partial class AppWindowPage : Page
{
    public AppWindow MyAppWindow { get; set; }

    // ...
}
```

MainPage.xaml.cs

In `MainPage`, get a reference to the page instance and assign the newly created `AppWindow` to the property in `AppWindowPage`.

```
C#

private async void ShowNewWindowButton_Click(object sender, RoutedEventArgs e)
{
    // Create a new window.
    AppWindow appWindow = await AppWindow.TryCreateAsync();

    // Create a Frame and navigate to the Page you want to show in the new window.
    Frame appWindowContentFrame = new Frame();
    appWindowContentFrame.Navigate(typeof(AppWindowPage));

    // Get a reference to the page instance and assign the
    // newly created AppWindow to the MyAppWindow property.
    AppWindowPage page = (AppWindowPage)appWindowContentFrame.Content;
    page.MyAppWindow = appWindow;

    // ...
}
```

Tracking app windows using UIContext

You might also want to have access to the [AppWindow](#) instances from other parts of your app. For example, `MainPage` could have a 'close all' button that closes all the tracked instances of `AppWindow`.

In this case, you should use the [UIContext](#) unique identifier to track the window instances in a [Dictionary](#).

MainPage.xaml.cs

In `MainPage`, create the Dictionary as a static property. Then, add the page to the Dictionary when you create it, and remove it when the page is closed. You can get the

UIContext from the content `Frame` (`appWindowContentFrame.UIContext`) after you call `ElementCompositionPreview.SetAppWindowContent`.

C#

```
public sealed partial class MainPage : Page
{
    // Track open app windows in a Dictionary.
    public static Dictionary<UIContext, AppWindow> AppWindows { get; set; }
        = new Dictionary<UIContext, AppWindow>();

    // ...

    private async void ShowNewWindowButton_Click(object sender,
RoutedEventArgs e)
    {
        // Create a new window.
        AppWindow appWindow = await AppWindow.TryCreateAsync();

        // Create a Frame and navigate to the Page you want to show in the
new window.
        Frame appWindowContentFrame = new Frame();
        appWindowContentFrame.Navigate(typeof(AppWindowPage));

        // Attach the XAML content to the window.
        ElementCompositionPreview.SetAppWindowContent(appWindow,
appWindowContentFrame);

        // Add the new page to the Dictionary using the UIContext as the
Key.
        AppWindows.Add(appWindowContentFrame.UIContext, appWindow);
        appWindow.Title = "App Window " + AppWindows.Count.ToString();

        // When the window is closed, be sure to release
// XAML resources and the reference to the window.
        appWindow.Closed += delegate
        {
            MainPage.AppWindows.Remove(appWindowContentFrame.UIContext);
            appWindowContentFrame.Content = null;
            appWindow = null;
        };

        // Show the window.
        await appWindow.TryShowAsync();
    }

    private async void CloseAllButton_Click(object sender, RoutedEventArgs
e)
    {
        while (AppWindows.Count > 0)
        {
            await AppWindows.Values.First().CloseAsync();
        }
    }
}
```

```
// ...  
}
```

AppWindowPage.xaml.cs

To use the [AppWindow](#) instance in your `AppWindowPage` code, use the page's [UIContext](#) to retrieve it from the static Dictionary in `MainPage`. You should do this in the page's [Loaded](#) event handler rather than in the constructor so that `UIContext` is not null. You can get the `UIContext` from the Page: `this.UIContext`.

C#

```
public sealed partial class AppWindowPage : Page  
{  
    AppWindow window;  
  
    // ...  
    public AppWindowPage()  
    {  
        this.InitializeComponent();  
  
        Loaded += AppWindowPage_Loaded;  
    }  
  
    private void AppWindowPage_Loaded(object sender, RoutedEventArgs e)  
    {  
        // Get the reference to this AppWindow that was stored when it was  
        // created.  
        window = MainPage.AppWindows[this.UIContext];  
  
        // Set up event handlers for the window.  
        window.Changed += Window_Changed;  
    }  
    // ...  
}
```

ⓘ Note

The `HelloAppWindow` example shows both ways to track the window in `AppWindowPage`, but you will typically use one or the other, not both.

Request window size and placement

The [AppWindow](#) class has several methods you can use to control the size and placement of the window. As implied by the method names, the system may or may not

honor the requested changes depending on environmental factors.

Call [RequestSize](#) to specify a desired window size, like this.

C#

```
colorPickerAppWindow.RequestSize(new Size(300, 428));
```

The methods to manage window placement are named *RequestMove**:

[RequestMoveAdjacentToCurrentView](#), [RequestMoveAdjacentToWindow](#),
[RequestMoveRelativeToDisplayRegion](#), [RequestMoveToDisplayRegion](#).

In this example, this code moves the window to be next to the main view that the window is spawned from.

C#

```
colorPickerAppWindow.RequestMoveAdjacentToCurrentView();
```

To get information about the current size and placement of the window, call [GetPlacement](#). This returns an [AppWindowPlacement](#) object that provides the current [DisplayRegion](#), [Offset](#), and [Size](#) of the window.

For example, you could call this code to move the window to the upper-right corner of the display. This code has to be called after the window has been displayed; otherwise, the window [Size](#) returned by the call to [GetPlacement](#) will be 0,0 and the offset will be incorrect.

C#

```
DisplayRegion displayRegion = window.GetPlacement().DisplayRegion;  
double displayRegionWidth = displayRegion.WorkAreaSize.Width;  
double windowWidth = window.GetPlacement().Size.Width;  
int horizontalOffset = (int)(displayRegionWidth - windowWidth);  
window.RequestMoveRelativeToDisplayRegion(displayRegion, new  
Point(horizontalOffset, 0));
```

Request a presentation configuration

The [AppWindowPresenter](#) class lets you show an [AppWindow](#) using a pre-defined configuration appropriate for the device it's shown on. You can use an [AppWindowPresentationConfiguration](#) value to place the window in `FullScreen` or `CompactOverlay` mode.

This example shows how to do the following:

- Use the [AppWindow.Changed](#) event to be notified if the available window presentations change.
- Use the [AppWindow.Presenter](#) property to get the current [AppWindowPresenter](#).
- Call [IsPresentationSupported](#) to see if a specific [AppWindowPresentationKind](#) is supported.
- Call [GetConfiguration](#) to check what kind of configuration is currently used.
- Call [RequestPresentation](#) to change the current configuration.

C#

```
private void Window_Changed(AppWindow sender, AppWindowChangedEventArgs args)
{
    if (args.DidAvailableWindowPresentationsChange)
    {
        EnablePresentationButtons(sender);
    }

    if (args.DidWindowPresentationChange)
    {
        ConfigText.Text =
window.Presenter.GetConfiguration().Kind.ToString();
    }

    if (args.DidSizeChange)
    {
        SizeText.Text = window.GetPlacement().Size.ToString();
    }
}

private void EnablePresentationButtons(AppWindow window)
{
    // Check whether the current AppWindowPresenter supports CompactOverlay.
    if
(window.Presenter.IsPresentationSupported(AppWindowPresentationKind.CompactO
verlay))
    {
        // Show the CompactOverlay button...
        compactOverlayButton.Visibility = Visibility.Visible;
    }
    else
    {
        // Hide the CompactOverlay button...
        compactOverlayButton.Visibility = Visibility.Collapsed;
    }

    // Check whether the current AppWindowPresenter supports FullScreen?
    if
(window.Presenter.IsPresentationSupported(AppWindowPresentationKind.FullScre
en))
```



```

    {
        // Show the FullScreen button...
        fullScreenButton.Visibility = Visibility.Visible;
    }
    else
    {
        // Hide the FullScreen button...
        fullScreenButton.Visibility = Visibility.Collapsed;
    }
}

private void CompactOverlayButton_Click(object sender, RoutedEventArgs e)
{
    if (window.Presenter.GetConfiguration().Kind !=
        AppWindowPresentationKind.CompactOverlay)
    {
        window.Presenter.RequestPresentation(AppWindowPresentationKind.CompactOverla
y);
        fullScreenButton.IsChecked = false;
    }
    else
    {
        window.Presenter.RequestPresentation(AppWindowPresentationKind.Default);
    }
}

private void FullScreenButton_Click(object sender, RoutedEventArgs e)
{
    if (window.Presenter.GetConfiguration().Kind !=
        AppWindowPresentationKind.FullScreen)
    {
        window.Presenter.RequestPresentation(AppWindowPresentationKind.FullScreen);
        compactOverlayButton.IsChecked = false;
    }
    else
    {
        window.Presenter.RequestPresentation(AppWindowPresentationKind.Default);
    }
}

```

Reuse XAML elements

An [AppWindow](#) lets you have multiple XAML trees with the same UI thread. However, a XAML element can only be added to a XAML tree once. If you want to move a part of your UI from one window to another, you have to manage it's placement in the XAML tree.

This example shows how to reuse a [ColorPicker](#) control while moving it between the main window and a secondary window.

The color picker is declared in the XAML for `MainPage`, which places it in the `MainPage` XAML tree.

XAML

```
<StackPanel x:Name="colorPickerContainer" Grid.Column="1"
Background="WhiteSmoke">
    <Button Click="DetachColorPickerButton_Click"
HorizontalAlignment="Right">
        <FontIcon FontFamily="Segoe MDL2 Assets" Glyph="" />
    </Button>
    <ColorPicker x:Name="colorPicker" Margin="12" Width="288"
        IsColorChannelTextInputVisible="False"
        ColorChanged="ColorPicker_ColorChanged"/>
</StackPanel>
```

When the color picker is detached to be placed in a new AppWindow, you first have to remove it from the `MainPage` XAML tree by removing it from its parent container. Though not required, this example also hides the parent container.

C#

```
colorPickerContainer.Children.Remove(colorPicker);
colorPickerContainer.Visibility = Visibility.Collapsed;
```

Then you can add it to the new XAML tree. Here, you first create a [Grid](#) that will be the parent container for the `ColorPicker`, and add the `ColorPicker` as a child of the `Grid`. (This lets you easily remove the `ColorPicker` from this XAML tree later.) You then set the `Grid` as the root of the XAML tree in the new window.

C#

```
Grid appWindowRootGrid = new Grid();
appWindowRootGrid.Children.Add(colorPicker);

// Create a new window
colorPickerAppWindow = await AppWindow.TryCreateAsync();

// Attach the XAML content to our window
ElementCompositionPreview.SetAppWindowContent(colorPickerAppWindow,
appWindowRootGrid);
```

When the [AppWindow](#) is closed, you reverse the process. First, remove the [ColorPicker](#) from the [Grid](#), then add it as a child of the [StackPanel](#) in `MainPage`.

C#

```
// When the window is closed, be sure to release XAML resources
// and the reference to the window.
colorPickerAppWindow.Closed += delegate
{
    appWindowRootGrid.Children.Remove(colorPicker);
    appWindowRootGrid = null;
    colorPickerAppWindow = null;

    colorPickerContainer.Children.Add(colorPicker);
    colorPickerContainer.Visibility = Visibility.Visible;
};
```

C#

```
private async void DetachColorPickerButton_Click(object sender,
RoutedEventArgs e)
{
    ColorPickerContainer.Visibility = Visibility.Collapsed;

    // Create the color picker window.
    if (colorPickerAppWindow == null)
    {
        ColorPickerContainer.Children.Remove(colorPicker);

        Grid appWindowRootGrid = new Grid();
        appWindowRootGrid.Children.Add(colorPicker);

        // Create a new window
        colorPickerAppWindow = await AppWindow.TryCreateAsync();
        colorPickerAppWindow.RequestMoveAdjacentToCurrentView();
        colorPickerAppWindow.RequestSize(new Size(300, 428));
        colorPickerAppWindow.Title = "Color picker";

        // Attach the XAML content to our window
        ElementCompositionPreview.SetAppWindowContent(colorPickerAppWindow,
appWindowRootGrid);

        // When the window is closed, be sure to release XAML resources
        // and the reference to the window.
        colorPickerAppWindow.Closed += delegate
        {
            appWindowRootGrid.Children.Remove(colorPicker);
            appWindowRootGrid = null;
            colorPickerAppWindow = null;

            ColorPickerContainer.Children.Add(colorPicker);
            ColorPickerContainer.Visibility = Visibility.Visible;
        }
    }
}
```

```

    };
}
// Show the window.
await colorPickerAppWindow.TryShowAsync();
}

```

Show a dialog box

By default, content dialogs display modally relative to the root [ApplicationView](#). When you use a [ContentDialog](#) inside an [AppWindow](#), you need to manually set the `XamlRoot` on the dialog to the root of the XAML host.

To do so, set the `ContentDialog`'s [XamlRoot](#) property to the same [XamlRoot](#) as an element already in the `AppWindow`. Here, this code is inside a button's [Click](#) event handler, so you can use the *sender* (the clicked `Button`) to get the `XamlRoot`.

C#

```

if
(ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContrac
t", 8))
{
    simpleDialog.XamlRoot = ((Button)sender).XamlRoot;
}

```

If you have one or more `AppWindows` open in addition to the main window (`ApplicationView`), each window can attempt to open a dialog, because the modal dialog will block only the window that it's rooted in. However, there can only be one [ContentDialog](#) open per thread at a time. Attempting to open two `ContentDialogs` will throw an exception, even if they are attempting to open in separate `AppWindows`.

To manage this, you should at the least open the dialog in a `try/catch` block to catch the exception in case another dialog is already open.

C#

```

try
{
    ContentDialogResult result = await simpleDialog.ShowAsync();
}
catch (Exception)
{
    // The dialog didn't open, probably because another dialog is already
    open.
}

```

Another way to manage dialogs is to track the currently open dialog, and close it before trying to open a new dialog. Here, you create a static property in `MainPage` called `CurrentDialog` for this purpose.

C#

```
public sealed partial class MainPage : Page
{
    // Track the last opened dialog so you can close it if another dialog
    // tries to open.
    public static ContentDialog CurrentDialog { get; set; } = null;

    // ...
}
```

Then, you check whether there is a currently opened dialog, and if there is, call the [Hide](#) method to close it. Finally, assign the new dialog to `CurrentDialog`, and try to show it.

C#

```
private async void DialogButton_Click(object sender, RoutedEventArgs e)
{
    ContentDialog simpleDialog = new ContentDialog
    {
        Title = "Content dialog",
        Content = "Dialog box for " + window.Title,
        CloseButtonText = "Ok"
    };

    if (MainPage.CurrentDialog != null)
    {
        MainPage.CurrentDialog.Hide();
    }
    MainPage.CurrentDialog = simpleDialog;

    // Use this code to associate the dialog to the appropriate AppWindow by
    // setting
    // the dialog's XamlRoot to the same XamlRoot as an element that is
    // already
    // present in the AppWindow.
    if
    (ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContrac
t", 8))
    {
        simpleDialog.XamlRoot = ((Button)sender).XamlRoot;
    }

    try
    {
        ContentDialogResult result = await simpleDialog.ShowAsync();
    }
}
```

```

        catch (Exception)
        {
            // The dialog didn't open, probably because another dialog is
            already open.
        }
    }
}

```

If it's not desirable to have a dialog closed programmatically, don't assign it as the `CurrentDialog`. Here, `MainPage` shows an important dialog that should only be dismissed when the user clicks `Ok`. Because it's not assigned as the `CurrentDialog`, no attempt is made to close it programmatically.

C#

```

public sealed partial class MainPage : Page
{
    // Track the last opened dialog so you can close it if another dialog
    tries to open.
    public static ContentDialog CurrentDialog { get; set; } = null;

    // ...
    private async void DialogButton_Click(object sender, RoutedEventArgs e)
    {
        ContentDialog importantDialog = new ContentDialog
        {
            Title = "Important dialog",
            Content = "This dialog can only be dismissed by clicking Ok.",
            CloseButtonText = "Ok"
        };

        if (MainPage.CurrentDialog != null)
        {
            MainPage.CurrentDialog.Hide();
        }
        // Do not track this dialog as the MainPage.CurrentDialog.
        // It should only be closed by clicking the Ok button.
        MainPage.CurrentDialog = null;

        try
        {
            ContentDialogResult result = await importantDialog.ShowAsync();
        }
        catch (Exception)
        {
            // The dialog didn't open, probably because another dialog is
            already open.
        }
    }
    // ...
}

```

Complete code

MainPage.xaml

XAML

```
<Page
    x:Class="HelloAppWindow.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:HelloAppWindow"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d"
    Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">

    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition/>
            <ColumnDefinition Width="Auto"/>
        </Grid.ColumnDefinitions>
        <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center">
            <Button x:Name="NewWindowButton" Content="Open new window"
                Click="ShowNewWindowButton_Click" Margin="0,12"/>
            <Button Content="Open dialog" Click="DialogButton_Click"
                HorizontalAlignment="Stretch"/>
            <Button Content="Close all" Click="CloseAllButton_Click"
                Margin="0,12" HorizontalAlignment="Stretch"/>
        </StackPanel>

        <StackPanel x:Name="colorPickerContainer" Grid.Column="1"
            Background="WhiteSmoke">
            <Button Click="DetachColorPickerButton_Click"
                HorizontalAlignment="Right">
                <FontIcon FontFamily="Segoe MDL2 Assets" Glyph="⦿" />
            </Button>
            <ColorPicker x:Name="colorPicker" Margin="12" Width="288"
                IsColorChannelTextInputVisible="False"
                ColorChanged="ColorPicker_ColorChanged"/>
        </StackPanel>
    </Grid>
</Page>
```

MainPage.xaml.cs

C#

```

using System;
using System.Collections.Generic;
using System.Linq;
using Windows.Foundation;
using Windows.UI;
using Windows.UI.WindowManagement;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Hosting;
using Windows.UI.Xaml.Media;

// The Blank Page item template is documented at
https://go.microsoft.com/fwlink/?LinkId=402352&clcid=0x409

namespace HelloAppWindow
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within a
    Frame.
    /// </summary>
    public sealed partial class MainPage : Page
    {
        AppWindow colorPickerAppWindow;

        // Track open app windows in a Dictionary.
        public static Dictionary<UIContext, AppWindow> AppWindows { get;
set; }

        = new Dictionary<UIContext, AppWindow>();

        // Track the last opened dialog so you can close it if another
        dialog tries to open.
        public static ContentDialog CurrentDialog { get; set; } = null;

        public MainPage()
        {
            this.InitializeComponent();
        }

        private async void ShowNewWindowButton_Click(object sender,
RoutedEventArgs e)
        {
            // Create a new window.
            AppWindow appWindow = await AppWindow.TryCreateAsync();

            // Create a Frame and navigate to the Page you want to show in
            the new window.
            Frame appWindowContentFrame = new Frame();
            appWindowContentFrame.Navigate(typeof(AppWindowPage));

            // Get a reference to the page instance and assign the
            // newly created AppWindow to the MyAppWindow property.
            AppWindowPage page =
(AppWindowPage)appWindowContentFrame.Content;
            page.MyAppWindow = appWindow;

```



```

        page.TextColorBrush = new SolidColorBrush(colorPicker.Color);

        // Attach the XAML content to the window.
        ElementCompositionPreview.SetAppWindowContent(appWindow,
appWindowContentFrame);

        // Add the new page to the Dictionary using the UIContext as the
Key.
        AppWindows.Add(appWindowContentFrame.UIContext, appWindow);
        appWindow.Title = "App Window " + AppWindows.Count.ToString();

        // When the window is closed, be sure to release XAML resources
        // and the reference to the window.
        appWindow.Closed += delegate
        {
            MainPage.AppWindows.Remove(appWindowContentFrame.UIContext);
            appWindowContentFrame.Content = null;
            appWindow = null;
        };

        // Show the window.
        await appWindow.TryShowAsync();
    }

    private async void DialogButton_Click(object sender, RoutedEventArgs
e)
    {
        ContentDialog importantDialog = new ContentDialog
        {
            Title = "Important dialog",
            Content = "This dialog can only be dismissed by clicking
Ok.",
            CloseButtonText = "Ok"
        };

        if (MainPage.CurrentDialog != null)
        {
            MainPage.CurrentDialog.Hide();
        }
        // Do not track this dialog as the MainPage.CurrentDialog.
        // It should only be closed by clicking the Ok button.
        MainPage.CurrentDialog = null;

        try
        {
            ContentDialogResult result = await
importantDialog.ShowAsync();
        }
        catch (Exception)
        {
            // The dialog didn't open, probably because another dialog
is already open.
        }
    }

```

```

        private async void DetachColorPickerButton_Click(object sender,
RoutedEventArgs e)
        {
            // Create the color picker window.
            if (colorPickerAppWindow == null)
            {
                colorPickerContainer.Children.Remove(colorPicker);
                colorPickerContainer.Visibility = Visibility.Collapsed;

                Grid appWindowRootGrid = new Grid();
                appWindowRootGrid.Children.Add(colorPicker);

                // Create a new window
                colorPickerAppWindow = await AppWindow.TryCreateAsync();
                colorPickerAppWindow.RequestMoveAdjacentToCurrentView();
                colorPickerAppWindow.RequestSize(new Size(300, 428));
                colorPickerAppWindow.Title = "Color picker";

                // Attach the XAML content to our window

ElementCompositionPreview.SetAppWindowContent(colorPickerAppWindow,
appWindowRootGrid);

                // Make sure to release the reference to this window,
                // and release XAML resources, when it's closed
                colorPickerAppWindow.Closed += delegate
                {
                    appWindowRootGrid.Children.Remove(colorPicker);
                    appWindowRootGrid = null;
                    colorPickerAppWindow = null;

                    colorPickerContainer.Children.Add(colorPicker);
                    colorPickerContainer.Visibility = Visibility.Visible;
                };
            }
            // Show the window.
            await colorPickerAppWindow.TryShowAsync();
        }

        private void ColorPicker_ColorChanged(ColorPicker sender,
ColorChangedEventArgs args)
        {
            NewWindowButton.Background = new SolidColorBrush(args.NewColor);
        }

        private async void CloseAllButton_Click(object sender,
RoutedEventArgs e)
        {
            while (AppWindows.Count > 0)
            {
                await AppWindows.Values.First().CloseAsync();
            }
        }
    }

```

```
}
```

AppWindowPage.xaml

XAML

```
<Page
    x:Class="HelloAppWindow.AppWindowPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:HelloAppWindow"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d"
    Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">

    <Grid>
        <TextBlock x:Name="TitleTextBlock" Text="Hello AppWindow!"
        FontSize="24" HorizontalAlignment="Center" Margin="24"/>

        <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center">
            <Button Content="Open dialog" Click="DialogButton_Click"
            Width="200" Margin="0,4"/>
            <Button Content="Move window" Click="MoveWindowButton_Click"
            Width="200" Margin="0,4"/>
            <ToggleButton Content="Compact Overlay"
            x:Name="compactOverlayButton" Click="CompactOverlayButton_Click"
            Width="200" Margin="0,4"/>
            <ToggleButton Content="Full Screen" x:Name="fullScreenButton"
            Click="FullScreenButton_Click"
            Width="200" Margin="0,4"/>
            <Grid>
                <TextBlock Text="Size:"/>
                <TextBlock x:Name="SizeText" HorizontalAlignment="Right"/>
            </Grid>
            <Grid>
                <TextBlock Text="Presentation:"/>
                <TextBlock x:Name="ConfigText" HorizontalAlignment="Right"/>
            </Grid>
        </StackPanel>
    </Grid>
</Page>
```

AppWindowPage.xaml.cs

C#

```

using System;
using Windows.Foundation;
using Windows.Foundation.Metadata;
using Windows.UI;
using Windows.UI.WindowManagement;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Media;

// The Blank Page item template is documented at
https://go.microsoft.com/fwlink/?LinkId=234238

namespace HelloAppWindow
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within a
    Frame.
    /// </summary>
    public sealed partial class AppWindowPage : Page
    {
        AppWindow window;

        public AppWindow MyAppWindow { get; set; }

        public SolidColorBrush TextColorBrush { get; set; } = new
        SolidColorBrush(Colors.Black);

        public AppWindowPage()
        {
            this.InitializeComponent();

            Loaded += AppWindowPage_Loaded;
        }

        private void AppWindowPage_Loaded(object sender, RoutedEventArgs e)
        {
            // Get the reference to this AppWindow that was stored when it
            was created.
            window = MainPage.AppWindows[this.UIContext];

            // Set up event handlers for the window.
            window.Changed += Window_Changed;

            TitleTextBlock.Foreground = TextColorBrush;
        }

        private async void DialogButton_Click(object sender, RoutedEventArgs
e)
        {
            ContentDialog simpleDialog = new ContentDialog
            {
                Title = "Content dialog",
                Content = "Dialog box for " + window.Title,
                CloseButtonText = "Ok"
            }
        }
    }
}

```

```

};

if (MainPage.CurrentDialog != null)
{
    MainPage.CurrentDialog.Hide();
}
MainPage.CurrentDialog = simpleDialog;

// Use this code to associate the dialog to the appropriate
AppWindow by setting
// the dialog's XamlRoot to the same XamlRoot as an element that
is already
// present in the AppWindow.
if
(ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContrac
t", 8))
{
    simpleDialog.XamlRoot = ((Button)sender).XamlRoot;
}

try
{
    ContentDialogResult result = await simpleDialog.ShowAsync();
}
catch (Exception)
{
    // The dialog didn't open, probably because another dialog
is already open.
}
}

private void Window_Changed(AppWindow sender,
AppWindowChangedEventArgs args)
{
    if (args.DidAvailableWindowPresentationsChange)
    {
        EnablePresentationButtons(sender);
    }

    if (args.DidWindowPresentationChange)
    {
        ConfigText.Text =
window.Presenter.GetConfiguration().Kind.ToString();
    }

    if (args.DidSizeChange)
    {
        SizeText.Text = window.GetPlacement().Size.ToString();
    }
}

private void EnablePresentationButtons(AppWindow window)
{
    // Check whether the current AppWindowPresenter supports
CompactOverlay.

```

```

        if
(window.Presenter.IsPresentationSupported(AppWindowPresentationKind.CompactO
verlay))
    {
        // Show the CompactOverlay button...
        compactOverlayButton.Visibility = Visibility.Visible;
    }
    else
    {
        // Hide the CompactOverlay button...
        compactOverlayButton.Visibility = Visibility.Collapsed;
    }

    // Check whether the current AppWindowPresenter supports
FullScreen?
    if
(window.Presenter.IsPresentationSupported(AppWindowPresentationKind.FullScre
en))
    {
        // Show the FullScreen button...
        fullScreenButton.Visibility = Visibility.Visible;
    }
    else
    {
        // Hide the FullScreen button...
        fullScreenButton.Visibility = Visibility.Collapsed;
    }
}

private void CompactOverlayButton_Click(object sender,
RoutedEventArgs e)
{
    if (window.Presenter.GetConfiguration().Kind !=
AppWindowPresentationKind.CompactOverlay)
    {

window.Presenter.RequestPresentation(AppWindowPresentationKind.CompactOverla
y);

        fullScreenButton.IsChecked = false;
    }
    else
    {

window.Presenter.RequestPresentation(AppWindowPresentationKind.Default);

    }
}

private void FullScreenButton_Click(object sender, RoutedEventArgs
e)
{
    if (window.Presenter.GetConfiguration().Kind !=
AppWindowPresentationKind.FullScreen)
    {

window.Presenter.RequestPresentation(AppWindowPresentationKind.FullScreen);

```

```

        compactOverlayButton.IsChecked = false;
    }
    else
    {
        window.Presenter.RequestPresentation(AppWindowPresentationKind.Default);
    }
}

private void MoveWindowButton_Click(object sender, RoutedEventArgs
e)
{
    DisplayRegion displayRegion =
window.GetPlacement().DisplayRegion;
    double displayRegionWidth = displayRegion.WorkAreaSize.Width;
    double windowWidth = window.GetPlacement().Size.Width;
    int horizontalOffset = (int)(displayRegionWidth - windowWidth);
    window.RequestMoveRelativeToDisplayRegion(displayRegion, new
Point(horizontalOffset, 0));
}
}
}

```

Related topics

- [Show multiple views](#)
- [Show multiple views with ApplicationView](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Show multiple views with ApplicationView

Article • 11/26/2024

Help your users be more productive by letting them view independent parts of your app in separate windows. When you create multiple windows for an app, each window behaves independently. The taskbar shows each window separately. Users can move, resize, show, and hide app windows independently and can switch between app windows as if they were separate apps. Each window operates in its own thread.

Important APIs: [ApplicationViewSwitcher](#), [CreateNewView](#)

What is a view?

An app view is the 1:1 pairing of a thread and a window that the app uses to display content. It's represented by a [Windows.ApplicationModel.Core.CoreApplicationView](#) object.

Views are managed by the [CoreApplication](#) object. You call [CoreApplication.CreateNewView](#) to create a [CoreApplicationView](#) object. The [CoreApplicationView](#) brings together a [CoreWindow](#) and a [CoreDispatcher](#) (stored in the [CoreWindow](#) and [Dispatcher](#) properties). You can think of the [CoreApplicationView](#) as the object that the Windows Runtime uses to interact with the core Windows system.

You typically don't work directly with the [CoreApplicationView](#). Instead, the Windows Runtime provides the [ApplicationView](#) class in the [Windows.UI.ViewManagement](#) namespace. This class provides properties, methods, and events that you use when your app interacts with the windowing system. To work with an [ApplicationView](#), call the static [ApplicationView.GetForCurrentView](#) method, which gets an [ApplicationView](#) instance tied to the current [CoreApplicationView](#)'s thread.

Likewise, the XAML framework wraps the [CoreWindow](#) object in a [Windows.UI.Xaml.Window](#) object. In a XAML app, you typically interact with the [Window](#) object rather than working directly with the [CoreWindow](#).

Show a new view

While each app layout is unique, we recommend including a "new window" button in a predictable location, such as the top right corner of the content that can be opened in a

new window. Also consider including a context menu option to "Open in a new window".

Let's look at the steps to create a new view. Here, the new view is launched in response to a button click.

C#

```
private async void Button_Click(object sender, RoutedEventArgs e)
{
    CoreApplicationView newView = CoreApplication.CreateNewView();
    int newViewId = 0;
    await newView.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        Frame frame = new Frame();
        frame.Navigate(typeof(SecondaryPage), null);
        Window.Current.Content = frame;
        // You have to activate the window in order to show it later.
        Window.Current.Activate();

        newViewId = ApplicationView.GetForCurrentView().Id;
    });
    bool viewShown = await
    ApplicationViewSwitcher.TryShowAsStandaloneAsync(newViewId);
}
```

To show a new view

1. Call `CoreApplication.CreateNewView` to create a new window and thread for the view content.

C#

```
CoreApplicationView newView = CoreApplication.CreateNewView();
```

2. Track the `Id` of the new view. You use this to show the view later.

You might want to consider building some infrastructure into your app to help with tracking the views you create. See the `ViewLifetimeControl` class in the [MultipleViews sample](#) [↗](#) for an example.

C#

```
int newViewId = 0;
```

3. On the new thread, populate the window.

You use the [CoreDispatcher.RunAsync](#) method to schedule work on the UI thread for the new view. You use a [lambda expression](#) to pass a function as an argument to the **RunAsync** method. The work you do in the lambda function happens on the new view's thread.

In XAML, you typically add a **Frame** to the **Window's Content** property, then navigate the **Frame** to a XAML **Page** where you've defined your app content. For more info about frames and pages, see [Peer-to-peer navigation between two pages](#).

After the new **Window** is populated, you must call the **Window's Activate** method in order to show the **Window** later. This work happens on the new view's thread, so the new **Window** is activated.

Finally, get the new view's **Id** that you use to show the view later. Again, this work is on the new view's thread, so [ApplicationView.GetForCurrentView](#) gets the **Id** of the new view.

C#

```
await newView.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
{
    Frame frame = new Frame();
    frame.Navigate(typeof(SecondaryPage), null);
    Window.Current.Content = frame;
    // You have to activate the window in order to show it later.
    Window.Current.Activate();

    newViewId = ApplicationView.GetForCurrentView().Id;
});
```

4. Show the new view by calling

[ApplicationViewSwitcher.TryShowAsStandaloneAsync](#).

After you create a new view, you can show it in a new window by calling the [ApplicationViewSwitcher.TryShowAsStandaloneAsync](#) method. The *viewId* parameter for this method is an integer that uniquely identifies each of the views in your app. You retrieve the view **Id** by using either the **ApplicationView.Id** property or the [ApplicationView.GetApplicationViewIdForWindow](#) method.

C#

```
bool viewShown = await
ApplicationViewSwitcher.TryShowAsStandaloneAsync(newViewId);
```

The main view

The first view that's created when your app starts is called the *main view*. This view is stored in the [CoreApplication.MainView](#) property, and its [IsMain](#) property is true. You don't create this view; it's created by the app. The main view's thread serves as the manager for the app, and all app activation events are delivered on this thread.

If secondary views are open, the main view's window can be hidden – for example, by clicking the close (x) button in the window title bar - but its thread remains active. Calling [Close](#) on the main view's [Window](#) causes an [InvalidOperationException](#) to occur. (Use [Application.Exit](#) to close your app.) If the main view's thread is terminated, the app closes.

Secondary views

Other views, including all views that you create by calling [CreateNewView](#) in your app code, are secondary views. Both the main view and secondary views are stored in the [CoreApplication.Views](#) collection. Typically, you create secondary views in response to a user action. In some instances, the system creates secondary views for your app.

ⓘ Note

You can use the Windows *assigned access* feature to run an app in [kiosk mode](#). When you do this, the system creates a secondary view to present your app UI above the lock screen. App-created secondary views are not allowed, so if you try to show your own secondary view in kiosk mode, an exception is thrown.

Switch from one view to another

Consider providing a way for the user to navigate from a secondary window back to its parent window. To do this, use the [ApplicationViewSwitcher.SwitchAsync](#) method. You call this method from the thread of the window you're switching from and pass the view ID of the window you're switching to.

C#

```
await ApplicationViewSwitcher.SwitchAsync(viewIdToShow);
```

When you use [SwitchAsync](#), you can choose if you want to close the initial window and remove it from the taskbar by specifying the value of

Related topics

- [Show multiple views](#)
- [Show multiple views with AppWindow](#)
- [ApplicationViewSwitcher](#)
- [CreateNewView](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Getting started

Article • 08/22/2024

This article is a getting started guide for creators who want to develop games on Windows or Xbox.

ⓘ Note

Before investing time developing a game based on the UWP framework, please note that UWP is no longer under active development. Further, UWP based games are no longer accepted in the Xbox Store.

Here are some questions to help you find the info you need:

- Are you an experienced game developer and want all the details? See the [Windows game development guide](#).
- Completely new to coding? Something fun like the [Minecraft Hour of Code tutorials](#) [↗] might be of interest.
- Just looking for great games to play? Check out the [Microsoft Store](#) [↗].

Quick start guide

Steps to get into developing games right away.

Step 1: Get the software and tools

Make sure you have Windows 10 installed on your device and have the latest updates installed.

Install a suitable IDE like Visual Studio. Visual Studio Community 2017 is available for free download. For more information, see [Visual Studio downloads](#) [↗].

If you plan to use a game engine and other middleware, see [Bridges, game engines, and middleware](#) section in the [Windows game development guide](#). For information about developing Windows games using a specific game engine, you'll need to go to the game engine's documentation.

Step 2: Prepare your hardware for development

If you're doing development for the first time, you'll need to enable Developer Mode on your device. For more information, see [Enable your device for development](#).

Step 3: Run a sample and see how it works

To get started with UWP DirectX development, see [Create a simple UWP game with DirectX](#). If you simply want to read and be acquainted with DirectX concepts like what a buffer is, see [Direct3D graphics concepts](#).

For more samples, see [Game samples](#).

Step 4: Sign up for a Partner Center account

A Partner Center account gives you access to [Partner Center](#) , which lets you manage and submit all of your apps and games for Windows devices in one place.

For Windows game development, you may choose to wait till you want access to Partner Center or when you want to use Xbox Live features in your game.

For more information, see [Publish Windows apps and games](#).

Useful links

- [Windows game development guide](#)
- [What's a UWP app?](#)
- [Using cloud services for UWP games](#)
- [Making games accessible](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Windows game development guide

Article • 08/22/2024

Welcome to the Windows game development guide!

ⓘ Note

Before investing time developing a game based on the UWP framework, please note that UWP is no longer under active development. Further, UWP based games are no longer accepted in the Xbox Store.

This guide provides an end-to-end collection of the resources and information you'll need to develop a Universal Windows Platform (UWP) game. An English (US) version of this guide is available in [PDF](#) [↗](#) format.

Introduction to game development for the Universal Windows Platform (UWP)

This guide provides an end-to-end collection of information and resources that will help you as you develop your game. The sections are organized according to the stages of game development, so you'll know where to look for information when you need it.

If you're new to developing games on Windows, the [Getting started](#) guide might be where you want to start off. The [Game development resources](#) section also provides a high-level survey of documentation, programs, and other resources that are helpful when creating a game. If you want to start by looking at some UWP code instead, see [Game samples](#).

Game development resources

From documentation to developer programs, forums, blogs, and samples, there are many resources available to help you on your game development journey. Here's a roundup of resources to know about as you begin developing your Windows game.

ⓘ Note

Some features are managed through various programs. This guide covers a broad range of resources, so you might find that some resources are inaccessible

depending on the program you're in, or your specific development role. For information about partnering with Microsoft, see [Developer programs](#).

Game development documentation

Throughout this guide, you'll find deep links to relevant documentation—organized by task, technology, and stage of game development. To give you a broad view of what's available, here are the main documentation portals for Windows game development.

 Expand table

Windows Dev Center main portal	Windows Dev Center
Developing Windows apps	Develop Windows apps
Universal Windows Platform app development	Universal Windows Platform documentation
How-to guides for UWP games	Games and DirectX
DirectX reference and overviews	DirectX Graphics and Gaming
Azure for gaming	Build and scale your games using Azure 
Azure PlayFab	Complete backend solution for live games 
Mixer Interactive documentation	Add interactivity to your game

Partner Center

[Registering as a developer in Partner Center](#) is the first step towards publishing your Windows game. A developer account lets you reserve your game's name, and submit free or paid games to the Microsoft Store for all Windows devices. Use your developer account to manage your game and in-game products, get detailed analytics, and enable services that create great experiences for your players around the world.

Microsoft also offers several developer programs to help you develop and publish Windows games. We recommend seeing whether any are right for you before registering for a Partner Center account. For more info, go to [Developer programs](#)

Developer programs

To publish a game in the Microsoft Store, you'll also need to create a developer account in [Partner Center](#)  .

Game samples

There are many Windows game and app samples available to help you understand Windows gaming features, and get a quick start on game development. Samples are developed and published regularly, so don't forget to occasionally check back on sample portals to see what's new. You can also [watch](#)  GitHub repos to be notified of changes and additions.

 Expand table

Universal Windows Platform sample apps	Windows-universal-samples 
Direct3D 12 graphics samples	DirectX-Graphics-Samples 
Direct3D 11 graphics samples	directx-sdk-samples 
Direct3D 11 first-person game sample	Create a simple UWP game with DirectX
Direct2D custom image effects sample	D2DCustomEffects 
Direct2D gradient mesh sample	D2DGradientMesh 
Direct2D photo adjustment sample	D2DPhotoAdjustment 
Windows game samples (MSDN Code Gallery)	Microsoft Store game samples
JavaScript 2D game sample	Create a UWP game in JavaScript
JavaScript 3D game sample	Creating a 3D JavaScript game using three.js

Developer forums

Developer forums are a great place to ask and answer game development questions, and connect with the game development community. Forums can also be fantastic resources for finding existing answers to difficult issues that developers have faced and solved in the past.

 Expand table

Publishing apps and games developer forums	Publishing and ads-in-apps 
UWP apps developer forum	Developing Universal Windows Platform apps
Desktop applications developer forums	Windows desktop applications forums 
DirectX Microsoft Store games (archived forum posts)	Building Microsoft Store games with DirectX (archived) 

Windows 10 managed partner developer forums	XBOX Developer Forums: Windows 10 ↗
PlayFab forums	PlayFab forums ↗

Developer blogs

Developer blogs are another great resource for the latest information about game development. You'll find posts about new features, implementation details, best practices, architecture background, and more.

 Expand table

Building apps for Windows (blog)	Building Apps for Windows ↗
Windows 10 (blog posts)	Posts in Windows 10 ↗
Visual Studio engineering team blog	The Visual Studio Blog ↗
Visual Studio developer tools blogs	Developer Tools Blogs ↗
Somasegar's developer tools blog	Somasegar's blog ↗
DirectX developer blog	DirectX Developer blog ↗
DirectX 12 introduction (blog post)	DirectX 12 ↗
Visual C++ tools team blog	Visual C++ team blog ↗
PIX team blog	Performance tuning and debugging for DirectX 12 games on Windows and Xbox ↗
Universal Windows App Deployment team blog	Build and deploy UWP apps team blog

Concept and planning

In the concept and planning stage, you decide what your game is going to be like and the technologies and tools you'll use to bring it to life.

Overview of game development technologies

When you start developing a game for the UWP you have multiple options available for graphics, input, audio, networking, utilities, and libraries.

If you've already decided on all the technologies you'll be using in your game, then you're set. If not, the [Game technologies for UWP apps](#) guide is an excellent overview of many of the technologies available, and is highly recommended reading to help you understand the options and how they fit together.

 [Expand table](#)

Survey of UWP game technologies	Game technologies for UWP apps
---------------------------------	--

Game planning

These are some high-level concept and planning topics to consider when planning for your game.

 [Expand table](#)

Make your game accessible	Accessibility for games
Build games using cloud	Cloud for games
Monetize your game	Monetization for games

Choosing your graphics technology and programming language

There are several programming languages and graphics technologies available for use in Windows games. The path you take depends on the type of game you're developing, the experience and preferences of your development studio, and specific feature requirements of your game. Will you use C#, C++, or JavaScript? DirectX, XAML, or HTML5?

DirectX

Microsoft DirectX is the choice to make for the highest-performance 2D and 3D graphics and multimedia.

DirectX 12 is faster and more efficient than any previous version. Direct3D 12 enables richer scenes, more objects, more complex effects, and full utilization of modern GPU hardware on Windows PCs and Xbox One.

If you want to use the familiar graphics pipeline of Direct3D 11, you'll still benefit from the new rendering and optimization features added to Direct3D 11.3. And, if you're a

tried-and-true desktop Windows API developer with roots in Win32, then you'll still have that option for your Windows game.

The extensive features and deep platform integration of DirectX provide the power and performance needed by the most demanding games.

 Expand table

DirectX for UWP development	DirectX programming
Tutorial: How to create a UWP DirectX game	Create a simple UWP game with DirectX
DirectX overviews and reference	DirectX Graphics and Gaming
Direct3D 12 programming guide and reference	Direct3D 12 Graphics
Graphics and DirectX 12 development videos (YouTube channel)	Microsoft DirectX 12 and Graphics Education 

XAML

XAML is an easy-to-use declarative UI language with convenient features like animations, storyboards, data binding, scalable vector-based graphics, dynamic resizing, and scene graphs. XAML works great for game UI, menus, sprites, and 2D graphics. To make UI layout easy, XAML is compatible with design and development tools like Expression Blend and Microsoft Visual Studio. XAML is commonly used with C#, but C++ is also a good choice if that's your preferred language or if your game has high CPU demands.

 Expand table

XAML platform overview	XAML platform
XAML UI and controls	Controls, layouts, and text

HTML 5

HyperText Markup Language (HTML) is a common UI markup language used for web pages, apps, and rich clients. Windows games can use HTML5 as a full-featured presentation layer with the familiar features of HTML, access to the Universal Windows Platform, and support for modern web features like AppCache, Web Workers, canvas, drag-and-drop, asynchronous programming, and SVG. Behind the scenes, HTML rendering takes advantage of the power of DirectX hardware acceleration, so you can still get the performance benefits of DirectX without writing any extra code. HTML5 is a

good choice if you're proficient with web development, porting a web game, or you want to use language and graphics layers that can be easier to approach than the other choices. HTML5 is used with JavaScript, but can also call into components created with C# or C++/CX.

 Expand table

HTML5 and Document Object Model information	HTML and DOM reference 
The HTML5 W3C Recommendation	HTML5 

Combining presentation technologies

The Microsoft DirectX Graphics Infrastructure (DXGI) provides interoperability and compatibility across multiple graphics technologies. For high-performance graphics, you can combine XAML and DirectX; using XAML for menus and other simple UI, and DirectX for rendering complex 2D and 3D scenes. DXGI also provides compatibility between Direct2D, Direct3D, DirectWrite, DirectCompute, and the Microsoft Media Foundation.

 Expand table

DirectX Graphics Infrastructure programming guide and reference	DXGI
Combining DirectX and XAML	DirectX and XAML interop

C++

[C++/WinRT](#) is a high-performance, low overhead language that provides the powerful combination of speed, compatibility, and platform access. C++/WinRT makes it easy to use all of the great gaming features in Windows, including DirectX and Xbox Live. You can also reuse existing C++ code and libraries. C++/WinRT creates fast, native code that doesn't incur the overhead of garbage collection, so your game can have great performance and low power consumption, which leads to longer battery life. Use C++/WinRT with DirectX or XAML, or create a game that uses a combination of both.

C#

C# (pronounced "C sharp") is a modern, innovative language that is simple, powerful, type-safe, and object-oriented. C# enables rapid development while retaining the

familiarity and expressiveness of C-style languages. Though easy to use, C# has numerous advanced language features such as polymorphism, delegates, lambdas, closures, iterator methods, covariance, and Language-Integrated Query (LINQ) expressions. C# is an excellent choice if you are targeting XAML, want to get a quick start developing your game, or have previous C# experience. C# is used primarily with XAML, so if you want to use DirectX, then choose C++ instead, or write part of your game as a C++ component that interacts with DirectX. Or, consider [Win2D](#): an immediate mode Direct2D graphics library for C# and C++.

 Expand table

C# programming guide and reference

[C# language reference](#)

Use Windows Runtime components to combine languages

With the Universal Windows Platform, it's easy to combine components written in different languages. Create Windows Runtime components in C++, C#, or Visual Basic, and then call into them from JavaScript, C#, C++, or Visual Basic. This is a great way to program portions of your game in the language of your choice. Components also let you consume external libraries that are only available in a particular language, as well as use legacy code you've already written.

 Expand table

How to create Windows Runtime components

[Windows Runtime components with C++/WinRT](#)

Which version of DirectX should your game use?

If you're choosing DirectX for your game, then you'll need to decide which version to use: Microsoft Direct3D 12 or Microsoft Direct3D 11.

DirectX 12 is faster and more efficient than any previous version. Direct3D 12 enables richer scenes, more objects, more complex effects, and full utilization of modern GPU hardware on Windows PCs and Xbox One. Since Direct3D 12 works at a very low level, it's able to give an expert graphics development team or an experienced DirectX 11 development team all the control they need to maximize graphics optimization.

Direct3D 11.3 is a low level graphics API that uses the familiar Direct3D programming model and handles for you more of the complexity involved in GPU rendering. It is also supported in Windows and Xbox One. If you have an existing engine written in Direct3D

11, and you're not quite ready to make the jump to Direct3D 12, you can use Direct3D 11 on 12 to achieve some performance improvements. Versions 11.3+ contain the new rendering and optimization features enabled also in Direct3D 12.

[Expand table](#)

Choosing Direct3D 12 or Direct3D 11	What is Direct3D 12?
Overview of Direct3D 11	Direct3D 11 Graphics
Overview of Direct3D 11 on 12	Direct3D 11 on 12

Bridges, game engines, and middleware

Depending on the needs of your game, using bridges, game engines, or middleware can save development and testing time and resources. Here are some overview and resources for bridges, game engines, and middleware.

Azure PlayFab

Now part of the Microsoft family, Azure PlayFab is a complete back-end platform for live games and a powerful way for independent studios to get started. Boost revenue, engagement, and retention—while cutting costs—with game services, real-time analytics, and LiveOps.

[Expand table](#)

PlayFab	Overview of tools and services ↗
Getting started	What is PlayFab?
Video tutorial series	Series of demo videos about PlayFab's core systems ↗
Recipes	Recipes
GitHub repo	Get scripts and SDKs for various platforms including Android, iOS, Windows, Unity, and Unreal. ↗
API documentation	REST API overview
Forums	PlayFab forums ↗

Unity

Unity offers a platform for creating beautiful and engaging 2D, 3D, VR, and AR games and apps. It enables you to realize your creative vision fast, and delivers your content to virtually any media or device.

Beginning with Unity 5.4, Unity supports Direct3D 12 development.

 Expand table

The Unity game engine	Unity - Game Engine 
Get Unity	Get Unity 
Unity documentation for Windows	Unity Manual / Windows 
Add LiveOps using PlayFab	Quickstart: PlayFab Client library for C# in Unity
How to add interactivity to your game using Mixer Interactive	Getting started guide 
Mixer SDK for Unity	Mixer Unity plugin 
Mixer SDK for Unity reference documentation	API reference for Mixer Unity plugin
Troubleshooting missing assembly references related to .NET APIs	Missing .NET APIs in Unity and UWP
Publish your Unity game as a Universal Windows Platform app (video)	How to publish your Unity game as a UWP app
Use Unity to make Windows games and apps (video)	Making Windows games and apps with Unity
Unity game development using Visual Studio (video series)	Using Unity with Visual Studio 2015 

Havok

Havok's modular suite of tools and technologies help game creators reach new levels of interactivity and immersion. Havok enables highly realistic physics, interactive simulations, and stunning cinematics. Version 2015.1 or later officially supports UWP in Visual Studio 2015 on x86, 64-bit, and Arm.

 Expand table

Havok website	Havok 
---------------	---

Unreal Engine

Unreal Engine is a complete suite of game development tools for all types of games and developers. For the most demanding console and PC games, Unreal Engine is used by game developers worldwide.

 Expand table

Unreal Engine overview	Unreal Engine 
------------------------	--

BabylonJS

BabylonJS is a complete JavaScript framework for building 3D games with HTML5, WebGL, WebVR, and Web Audio.

 Expand table

BabylonJS	BabylonJS 
Building a cross-platform WebGL game with BabylonJS	Use BabylonJS to develop a cross-platform game 

Prototype and design

Now that you've decided the type of game you want to create, and the tools and graphics technology you'll use to build it, you're ready to get started with the design and prototype. At its core, your game is a Universal Windows Platform app, so that's where you'll begin.

Introduction to the Universal Windows Platform (UWP)

The Universal Windows Platform (UWP) provides a common API platform across Windows devices. Games that target the UWP can call WinRT APIs that are common to all devices. Because the UWP provides guaranteed API layers, you can choose to create a single app package that will install across Windows devices. And if you want to, your game can still call APIs (including some classic Windows APIs from Win32 and .NET) that are specific to the devices your game runs on.

The following are excellent guides that discuss the Universal Windows Platform apps in detail, and are recommended reading to help you understand the platform.

[Expand table](#)

Introduction to Universal Windows Platform apps

[What's a Universal Windows Platform app?](#)

Getting started with UWP development

Getting set up and ready to develop a Universal Windows Platform app is quick and easy. The following guides take you through the process step-by-step.

[Expand table](#)

Getting started with UWP development

[Get started with Windows apps](#)

Getting set up for UWP development

[Get set up](#)

If you're an absolute beginner to UWP programming, and are considering using XAML in your game (see [Choosing your graphics technology and programming language](#)), the Windows 10 development for absolute beginners video series is a good place to start.

[Expand table](#)

Announcing the Windows 10 absolute beginners series using XAML (blog post)

[Windows 10 development for absolute beginners](#) [↗](#)

UWP development concepts

[Expand table](#)

Overview of Universal Windows Platform app development

[Develop Windows apps](#)

Overview of network programming in the UWP

[Networking and web services](#)

Using Windows.Web.HTTP and Windows.Networking.Sockets in games

[Networking for games](#)

Asynchronous programming concepts in the UWP

[Asynchronous programming](#)

Windows Desktop APIs and UWP

These are some links to help you interoperate between desktop and UWP code.

[Expand table](#)

Use existing C++ code for UWP game development	How to: Use existing C++ code in a UWP app
Windows Runtime APIs for Win32 and COM APIs	Win32 and COM APIs for UWP apps
Unsupported CRT functions in UWP	CRT functions not supported in Universal Windows Platform apps
Alternatives for Windows APIs	Alternatives to Windows APIs in Universal Windows Platform (UWP) apps

Process lifetime management

Process lifetime management, or app lifecycle, describes the various activation states that a Universal Windows Platform app can transition through. Your game can be activated, suspended, resumed, or terminated, and can transition through those states in a variety of ways.

[Expand table](#)

Handling app lifecycle transitions	App lifecycle
Using Microsoft Visual Studio to trigger app transitions	How to trigger suspend, resume, and background events for UWP apps in Visual Studio

Designing game UX

The genesis of a great game is inspired design.

Games share some common user interface elements and design principles with apps; but games often have a unique look, feel, and design goal for their user experience. Games succeed when thoughtful design is applied to both aspects—when should your game use tested UX, and when should it diverge and innovate? The presentation technology that you choose for your game—DirectX, XAML, HTML5, or some combination of the three—will influence implementation details, but the design principles you apply are largely independent of that choice.

Separately from UX design, gameplay design such as level design, pacing, world design, and other aspects is an art form of its own—one that's up to you and your team, and not covered in this development guide.

[Expand table](#)

UWP design basics and guidelines	Designing UWP apps
Designing for app lifecycle states	UX guidelines for launch, suspend, and resume

Color guideline and palette

Following a consistent color guideline in your game improves aesthetics, aids navigation, and is a powerful tool to inform the player of menu and HUD functionality. Consistent coloring of game elements like warnings, damage, XP, and achievements can lead to cleaner UI, and reduce the need for explicit labels.

[Expand table](#)

Color guide	Color in Windows 11
-------------	-------------------------------------

Typography

The appropriate use of typography enhances many aspects of your game, including UI layout, navigation, readability, atmosphere, brand, and player immersion.

[Expand table](#)

Typography guide	Best Practices: Typography
------------------	--

UI map

A UI map is a layout of game navigation and menus expressed as a flowchart. The UI map helps all involved stakeholders understand the game's interface and navigation paths, and can expose potential roadblocks and dead ends early in the development cycle.

[Expand table](#)

UI map guide	Best Practices: UI Map
--------------	--

Game audio

Here are guides and references for implementing audio in games using XAudio2, XAPO, and Windows Sonic. XAudio2 is a low-level audio API that provides signal processing

and mixing foundation for developing high-performance audio engines. The XAPO API allows the creation of cross-platform audio processing objects (XAPO) for use in XAudio2 on both Windows and Xbox. Windows Sonic audio support allows you to add Dolby Atmos for Home Theater, Dolby Atmos for Headphones, and Windows HRTF support to your game or streaming media application.

 [Expand table](#)

XAudio2 APIs	Programming guide and API reference for XAudio2
Create cross-platform audio processing objects	XAPO Overview
Intro to audio concepts	Audio for games
Windows Sonic overview	Spatial sound

Direct3D 12

Learn what's new and different in Direct3D 12 (compared to Direct3D 11), and how to start programming using Direct3D 12.

 [Expand table](#)

Direct3D 12 programming guide and reference	Direct3D 12 graphics
Set up programming environment	Direct3D 12 programming environment setup
How to create a basic component	Creating a basic Direct3D 12 component
Changes in Direct3D 12	Important changes migrating from Direct3D 11 to Direct3D 12
How to port from Direct3D 11 to Direct3D 12	Porting from Direct3D 11 to Direct3D 12
Resource binding concepts (covering descriptor, descriptor table, descriptor heap, and root signature)	Resource binding in Direct3D 12
Managing memory	Memory management in Direct3D 12

DirectX development

Guides and references for DirectX game development.

[Expand table](#)

DirectX for UWP development	DirectX programming
Tutorial: How to create a UWP DirectX game	Create a simple UWP game with DirectX
DirectX interaction with the UWP app model	The app object and DirectX
DirectX overviews and reference	DirectX Graphics and Gaming
DirectX 12 fundamentals (video)	Better Power, Better Performance: Your Game on DirectX 12

DirectX Tool Kit and libraries

The DirectX Tool Kit, DirectX texture processing library, DirectXMesh geometry processing library, UVAtlas library, and DirectXMath library provide texture, mesh, sprite, and other utility functionality and helper classes for DirectX development. These libraries can help save you development time and effort.

[Expand table](#)

Get DirectX Tool Kit for DirectX 12	DirectXTK 12 ↗
Get DirectX Tool Kit for DirectX 11	DirectXTK ↗
Get DirectX texture processing library	DirectXTex ↗
Get DirectXMesh geometry processing library	DirectXMesh ↗
Get UVAtlas for creating and packing isochart texture atlas	UVAtlas ↗
Get the DirectXMath library	DirectXMath ↗
Direct3D 12 support in the DirectXTK (GitHub issue)	Support for DirectX 12 ↗

DirectX resources from partners

Here's some additional DirectX documentation created by external partners.

[Expand table](#)

Nvidia: DX12 Do's and Don'ts (blog post)	DX12 Do's and Don'ts, Updated! ↗
Intel: Parallel Processing with DirectX 3D 12	Parallel Processing with DirectX 3D 12 ↗

Intel: How to implement an explicit multi-adapter application using DirectX 12	Multi-Adapter Support in DirectX 12 
Intel: Collaborative white paper by Intel, Suzhou Snail, and Microsoft	Tutorial: Migrating Your Apps to DirectX 12 – Part 1 

Production

At this stage, your studio is fully engaged and moving into the production cycle, with work distributed throughout your team. You're polishing, refactoring, and extending the prototype to craft it into a full game.

Notifications and live tiles

A tile is your game's representation on the Start Menu. Tiles and notifications can drive player interest even when they aren't currently playing your game.

 Expand table

Developing tiles and badges	Tiles, badges, and notifications
Sample illustrating live tiles and notifications	Notifications sample 
Adaptive tile templates (blog post)	Adaptive Tile Templates - Schema and Documentation
Designing tiles and badges	Guidelines for tiles and badges
Windows 10 app for interactively developing live tile templates	Notifications Visualizer 
UWP Tile Generator extension for Visual Studio	Tool for creating all required tiles using single image 
UWP Tile Generator extension for Visual Studio (blog post)	Tips on using the UWP Tile Generator tool 

Enable in-app product (add-on) purchases

An add-on (in-app product) is a supplementary item that players can purchase in-game. Add-ons can be game levels, items, or anything else that your players might enjoy. Used appropriately, add-ons can provide revenue while improving the game experience. You define and publish your game's add-ons through Partner Center, and enable in-app purchases in your game's code.

[Expand table](#)

Durable add-ons	Enable in-app product purchases
Consumable add-ons	Enable consumable in-app product purchases
Add-on details and submission	Add-on submissions
Monitor add-on sales and demographics for your game	Add-on acquisitions report

Debugging, performance optimization, and monitoring

To optimize performance, you can take advantage of Game Mode in Windows to provide your gamers with the best possible gaming experience by fully utilizing the capacity of their current hardware.

The Windows Performance Toolkit (WPT) consists of performance monitoring tools that produce in-depth performance profiles of Windows operating systems and applications. This is especially useful for monitoring memory usage, and improving game performance. The Windows Performance Toolkit is included in the Windows SDK and Windows ADK. This toolkit consists of two independent tools: Windows Performance Recorder (WPR) and Windows Performance Analyzer (WPA). ProcDump, which is part of [Windows Sysinternals](#), is a command-line utility that monitors CPU spikes and generates dump files during game crashes.

[Expand table](#)

Performance test your code	Azure Test Plans ¹
Improve performance by getting exclusive or priority access to hardware resources using Game Mode APIs	Game Mode
Get the Windows Performance Toolkit (WPT)	Windows Performance Toolkit
Get Windows Performance Toolkit (WPT) from Windows ADK	Windows ADK
Get ProcDump	ProcDump

Advanced DirectX techniques and concepts

Some portions of DirectX development can be nuanced and complex. When you get to the point in production where you need to dig down into the details of your DirectX

engine, or debug difficult performance problems, the resources and information in this section can help.

 Expand table

Performance tuning and debugging tool for DirectX 12 on Windows	Introducing PIX on Windows 
Direct3D 12 programming guide	Direct3D 12 graphics
Combining DirectX and XAML	DirectX and XAML interop

High dynamic range (HDR) content development

Build game content that uses the full color capabilities of HDR.

 Expand table

Learn how to render HDR content and detect whether the current display supports it	Direct3D 12 HDR sample 
Create and configure an advanced color using DirectX	Direct2D advanced color image rendering sample 

Globalization and localization

Develop World-ready games for the Windows platform, and learn about the international features built into Microsoft's top products.

 Expand table

Preparing your game for the global market	Globalization and localization
---	--

Submitting and publishing your game

The following guides and information help make the publishing and submission process go as smoothly as possible.

Publishing

You'll use [Partner Center](#)  to publish and manage your game packages.

[Expand table](#)

Partner Center app publishing	Publish Windows apps
Use Azure Active Directory (AAD) to add users to your Partner Center account	Create customer groups
Rating your game (blog post)	Now Available: Single age rating system to simplify app submissions ↗

Packaging and uploading

[Expand table](#)

Divide and group content to enable streaming install	UWP App Streaming install
Create optional packages like DLC game content	Optional packages and related set authoring
Package your UWP game	Packaging apps
Package your UWP DirectX game	Package your UWP DirectX game
Packaging your game as a 3rd party developer (blog post)	Create uploadable packages without publisher's store account access ↗
Creating app packages and app package bundles using MakeAppx	Create packages using app packager tool MakeAppx.exe
Signing your files digitally using SignTool	Sign files and verify signatures in files using SignTool
Uploading and versioning your game	Upload app packages

Policies and certification

Don't let certification issues delay your game's release. Here are policies and common certification issues to be aware of.

[Expand table](#)

Microsoft Store App Developer Agreement	App Developer Agreement
Policies for publishing apps in the Microsoft Store	Microsoft Store Policies

How to avoid some common app certification issues

[Avoid common certification failures](#)

Store manifest (StoreManifest.xml)

The store manifest (`StoreManifest.xml`) is an optional configuration file that you can include in your app package. The store manifest provides additional features that are not part of the `AppxManifest.xml` file. For example, you can use the store manifest to block installation of your game if a target device doesn't have the specified minimum DirectX feature level or the specified minimum system memory.

[Expand table](#)

Store manifest schema

[StoreManifest schema \(Windows 10\)](#)

Game lifecycle management

After you've finished development and shipped your game, it's not *game over*. You might be done with development on version 1, but your game's journey in the marketplace has only just begun. You'll want to monitor usage and error reporting, respond to user feedback, and publish updates to your game.

Partner Center analytics and promotion

[Expand table](#)

Partner Center analytics

[Analyze app performance](#)

Responding to customer reviews

[Respond to customer reviews](#)

Ways to promote your game

[Attract customers and promote your apps](#)

Visual Studio Application Insights

Visual Studio Application Insights provides performance, telemetry, and usage analytics for your published game. Application Insights helps you to detect and solve issues after your game is released, continuously monitor and improve usage, and understand how players are continuing to interact with your game. Application Insights works by adding an SDK into your app, which sends telemetry to the [Azure portal](#) [↗](#).

 Expand table

Application performance and usage analytics	Application Insights overview
Azure Monitor	Azure Monitor overview

Third party solutions for analytics and promotion

 Expand table

Understand player behavior using GameAnalytics	GameAnalytics 
Connect your UWP game to Google Analytics	Get Windows SDK for Google Analytics 
Use Facebook App Installs Ads to promote your game to Facebook users	Get Windows SDK for Facebook 
Use Vungle to add video ads into your games	Download Vungle SDK for Windows 

Creating and managing content updates

To update your published game, you submit a new app package with a higher version number. After the package makes its way through submission and certification, it will automatically be available to customers as an update.

 Expand table

Updating and versioning your game	Package version numbering
Game package management guidance	Guidance for app package management

Adding Xbox Live to your game

Xbox Live is a premier gaming network that connects millions of gamers across the world. Developers gain access to Xbox Live features that can organically grow their game's audience, including Xbox Live presence, Leaderboards, Cloud Saves, Game Hubs, Clubs, Party Chat, Game DVR, and more.

Note

If you would like to develop Xbox Live enabled titles, there are several options available to you. For info about the various programs, see [Choosing an Xbox Live](#)

[developer program.](#)

 Expand table

Xbox Live overview

[Xbox Live documentation](#)

For developers in the Xbox Live Creators Program

 Expand table

Overview

[Joining the Creators Program](#)

For managed partners and developers in the ID@Xbox program

 Expand table

Overview

[Getting started with Xbox Live, for Managed Partners](#)

Samples

[Xbox game development samples](#)

Additional resources

 Expand table

Videos from major conferences such as GDC and //build

[Game development videos](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Game programming

Article • 10/20/2022

The Universal Windows Platform (UWP) offers new opportunities to create, distribute, and monetize games. Learn about creating a game for Windows 10.

Topic	Description
Windows game development guide	An end-to-end guide with resources and information for developing UWP games.
Planning	This topic contains a list of articles for the game planning stage.
UWP programming	Learn how to use Windows Runtime APIs to develop UWP games.
Game development videos	A collection of game dev videos from major conferences and events.

To learn about developing UWP games using DirectX, go to [DirectX programming](#).

To make the best use of the game development overviews and tutorials, we recommend that you're familiar with these subjects.

- [C++/WinRT](#) is the language projection that we recommend for developing UWP games with DirectX. C++/WinRT is a standard modern C++17 language projection for Windows Runtime (WinRT) APIs. It supersedes Microsoft C++ with Component Extensions (C++/CX). For a code example, using C++/WinRT to call DirectX APIs, see [Full source code listing of a minimal Direct2D application](#).
- Basic graphics programming terminology.
- Basic Windows programming concepts.
- Basic familiarity with the Direct3D 9 (or later) APIs.

Planning for UWP games

Article • 10/20/2022

This section provides information about planning for your UWP game.

Topic	Description
Game technologies	A list of technologies to help you develop games.
Accessibility for games	Learn to make games more accessible.
Cloud for games	Use cloud for games.
Monetization for games	Ways to monetize your game.
Package your game	Prepare your game package for the Microsoft Store.
Concept approval	Get your game concept approved.

Game technologies for UWP apps

Article • 10/20/2022

In this guide, you'll learn about the technologies available for developing Universal Windows Platform (UWP) games.

Benefits of Windows 10 for game development

With the introduction of UWP in Windows 10, your Windows 10 titles will be able to span all of the Microsoft platforms. With free migration from previous versions of Windows, there is a steadily increasing number of Windows 10 clients. The combination of these two things means that your Windows 10 titles will be able to reach a huge number of customers through the Microsoft Store.

In addition, Windows 10 offers many new features that are particularly beneficial to games:

- Reduced memory paging and reduced overall memory system size
- Improved graphics memory management actively allocates and protects more memory for the foreground game

UWP games with C++ and DirectX

Real-time games requiring high performance should make use of the DirectX APIs. DirectX is a collection of native APIs for creating games and multimedia applications that require high performance, such as 3D games.

Development environment

To create games for UWP, you'll need to set up your development environment by installing Visual Studio 2015 or later. We recommend that you install the latest version of Visual Studio, giving you access to the latest development and security updates. Visual Studio allows you to create UWP apps and provides tools for game development:

- Visual Studio tools for DX game programming - Visual Studio provides tools for creating, editing, previewing, and exporting image, model, and shader resources. There are also tools that you can use to convert resources at build time and debug DirectX graphics code. For more information, see [Use Visual Studio tools for game programming](#).

- Visual Studio graphics diagnostics features - Graphics diagnostic tools are now available from within Windows as an optional feature. The diagnostic tools allow you to do graphics debugging, graphics frame analysis, and monitor GPU usage in real time. For more information, see [Use the DirectX runtime and Visual Studio graphics diagnostic features](#).

For more information, see [Prepare your Universal Windows Platform and DirectX programming](#).

Getting started with DirectX game project templates

After setting up your development environment, you can use one of the DirectX related project templates to create your UWP DirectX game. Visual Studio 2015 has three templates available for creating new UWP DirectX projects, **DirectX 11 App (Universal Windows)**, **DirectX 12 App (Universal Windows)**, and **DirectX 11 and XAML App (Universal Windows)**. For more information, see [Create a Universal Windows Platform and DirectX game project from a template](#).

Windows 10 APIs

Windows 10 provides an extensive collection of APIs that are useful for game development. There are APIs for almost all aspects of games including, 3D Graphics, 2D Graphics, Audio, Input, Text Resources, User Interface, and networking.

There are many APIs related to game development, but not all games need to use all of the APIs. For example, some games will only use 3D graphics and only make use of Direct3D, some games may only use 2D graphics and only make use of Direct2D, and still other games may make use of both. The following diagram shows the game development related APIs grouped by functionality type.



- 3D Graphics - Windows 10 supports two 3D graphics API sets, Direct3D 11, and [Direct3D 12](#). Both of these APIs provide the capability to create 3D and 2D

graphics. Direct3D 11 and Direct3D 12 are not used together, but either can be used with any of the APIs in the 2D Graphics and UI group. For more information about using the graphics APIs in your game, see [Basic 3D graphics for DirectX games](#).

 Expand table

API	Description
Direct3D 12	Direct3D 12 introduces the next version of Direct3D, the 3D graphics API at the heart of DirectX. This version of Direct3D is designed to be faster and more efficient than previous versions of Direct3D. The tradeoff for Direct3D 12's increased speed is that it is lower level and requires you to manage your graphics resources yourself and have more extensive graphics programming experience to realize the increased speed. When to use Use Direct3D 12 when you need to maximize your game's performance and your game is CPU bound. For more information See the Direct3d 12 documentation.
Direct3D 11	Direct3D 11 is the previous version of Direct3D and allows you to create 3D graphics using a higher level of hardware abstraction than D3D 12. When to use Use Direct3D 11 if you have existing Direct3D 11 code, your game is not CPU bound, or you want the benefit of having resources managed for you. For more information See the Direct3D 11 documentation.

- 2D Graphics and UI - APIs concerning 2D graphics such as text and user interfaces. All of the 2D graphics and UI APIs are optional.

API	Description
Direct2D	Direct2D is a hardware-accelerated, immediate-mode, 2-D graphics API that provides high performance and high-quality rendering for 2-D geometry, bitmaps, and text. The Direct2D API is built on Direct3D and is designed to interoperate well with GDI, GDI+, and Direct3D. When to use Direct2D can be used instead of Direct3D to provide graphics for pure 2D games such as a side-scroller or board game, or can be used with Direct3D to simplify creation of 2D graphics in a 3D game, such as a user interface or heads-up-display. For more information See the Direct2D documentation.
DirectWrite	DirectWrite provides extra capabilities for working with text and can be used with Direct3D or Direct2D to provide text output for user interfaces or other areas where text is required. DirectWrite supports measuring, drawing, and hit-testing of multi-format text. DirectWrite handles text in all supported languages for global and localized applications. DirectWrite also provides a low-level glyph rendering API for developers who want to perform their own layout and Unicode-to-glyph processing. When to use For more information See the DirectWrite documentation.
DirectComposition	DirectComposition is a Windows component that enables high-performance bitmap composition with transforms, effects, and animations. Application developers can use the DirectComposition API to create visually engaging user interfaces that feature rich

API	Description
	and fluid animated transitions from one visual to another. When to use DirectComposition is designed to simplify the process of composing visuals and creating animated transitions. If your game requires complex user interfaces, you can use DirectComposition to simplify the creation and management of the UI. For more information See the DirectComposition documentation.

- Audio - APIs concerning playing audio and applying audio effects. For information about using the audio APIs in your game, see [Audio for games](#).

 Expand table

API	Description
XAudio2	XAudio2 is a low-level audio API that provides a foundation for signal processing and mixing. XAudio is designed to be very responsive for game audio engines while maintaining the ability to create custom audio effects and complex chains of audio effects and filters. When to use Use XAudio2 when your game needs to play sounds with minimal overhead and delay. For more information See the XAudio2 documentation.
Audio graphs	For functionality that you can implement with XAudio2, you have the alternative of using the Windows Runtime audio graph APIs instead. To help you decide between the two alternatives, see Choosing Windows Runtime AudioGraph or XAudio2.

API	Description
	When to use Use audio graphs when your game needs to play sounds with minimal overhead and delay, but with a significantly easier-to-use API than XAudio2, and with the option of C# support. For more information See the Audio graphs documentation.
Media Foundation	Microsoft Media Foundation is designed for the playback of media files and streams, both audio and video, but can also be used in games when higher level functionality than XAudio2 is required and some additional overhead is acceptable. When to use Media foundation is particularly useful for cinematic scenes or non-interactive components of your game. Media foundation is also useful for decoding audio files for playback using XAudio2. For more information See the Microsoft Media Foundation overview.

- Input - APIs concerning input from the keyboard, mouse, gamepad, and other user input sources.

[Expand table](#)

API	Description
XInput	The XInput Game Controller API enables applications to receive input from game controllers. When to use If your game needs to support gamepad input and you have existing XInput code,